AUDIT REPORT

Burnt Labs

Marketplace & Asset Contracts

Prepared by SCV-Security

On 15/12/2025

Table of Contents

Table of Contents **2**
Introduction **3**
Scope Functionality 3
Submitted Codebase 3
Revisions – Submitted Codebase 4
Methodologies 4
Code Criteria 5
Findings Summary **6**
Findings Technical Details **7**
1. Pending funds cannot be refunded to the buyer 7
2. Pending sales created from FinalizeFor cause the NFT to be transferred incorrectly 9
3. Reserved NFT expiration requires a full reservation period wait, causing a denial of service 10
4. Missing denomination validation during the minimum price check 11
5. Incorrect fee calculation in pending sale finalization 12
6. Marketplace fees are incorrectly sent to the manager 13
7. Exact price plugins overwrite the listing price 14
8. Zero-price listings and rounding errors allow royalty fee bypass 15
9. Weak replay attack defense for pending sales 16
10. Missing validation in contract initialization 17
11. Two-step ownership transfer is not implemented 18
12. Redundant listing save operation 19
13. Redundant config load 20
14. Missing sale approvals in event emissions 21
15. Inconsistent helper function usage 22
Document Control **23**
Appendices **24**
A. Appendix - Risk assessment methodology 24
B. Appendix - Report Disclaimer 25

Introduction

SCV has been engaged by Burnt Labs to conduct a comprehensive security review with the goal of identifying potential security threats and vulnerabilities within the codebase. The purpose of this audit is to evaluate the security posture of the codebase and provide actionable recommendations to mitigate any identified risks. This report presents an overview of the findings from our security audit, outlining areas of concern and proposing effective measures to enhance the codebase's security.

Scope Functionality

The asset contract is an NFT contract built on top of CW721 that adds marketplace functionality through a plugin system, allowing NFT owners to list tokens for sale with customizable pricing, reservation mechanics, and collection-level enforcement rules.

The marketplace contract provides a centralized trading platform that manages listings across multiple asset contract collections and supports both immediate purchases and an optional manager-approval workflow, in which buyer funds are escrowed as pending sales. It also enables offer-based trading, where buyers can make offers on specific NFTs or entire collections that sellers can accept.

Submitted Codebase

Burnt Labs	
Repository	https://github.com/burnt-labs/contracts
Contracts	`asset` & `marketplace`
Commit	`e840a0336228b1181804db5fd949ad5ba2676d6d`
Branch	`feat/xion-asset`

Revisions – Submitted Codebase

Burnt Labs	
Repository	https://github.com/burnt-labs/contracts
Contracts	`asset` & `marketplace`
Commit	`e3dadc6b8141f7bc669dde9cde3fa9ed8315983a`
Branch	`fix/audit-fixes`

Methodologies

SCV performs a combination of automated and manual security testing based on the scope of testing. The testing performed is based on the extensive experience and knowledge of the auditor to provide the greatest coverage and value to Burnt Labs Testing includes, but is not limited to, the following:

- Understanding the application and its functionality purpose.
- Deploying SCV in-house tooling to automate dependency analysis and static code review.
- Analyse each line of the code base and inspect application security perimeter.
- Review underlying infrastructure technologies and supply chain security posture.

Code Criteria

This section provides an evaluation of specific criteria aspects as described below:

- **Documentation:** Evaluating the presence and comprehensiveness of publicly available or provided explanatory information, diagram flowcharts, comments, and supporting documents to enhance code understanding.
- **Coverage:** Evaluating whether the code adequately addresses all necessary cases and scenarios, ensuring that the intended functionality or requirements are sufficiently covered.
- **Readability:** Assessing how easily the code can be understood and maintained, considering factors such as code structure, naming conventions, and overall organisation.
- **Complexity:** Evaluating the complexity of the code, including factors such as, number of lines, conditional statements, and nested structures.

The status of each criteria is categorised as either **SUFFICIENT** or **NOT-SUFFICIENT** based on the audit assessment. This categorisation provides insights to identify areas that may require further attention and improvement.

Criteria	Status	Notes
Documentation	**SUFFICIENT**	N/A
Coverage	**SUFFICIENT**	N/A
Readability	**SUFFICIENT**	The codebase had good readability overall and utilised many Rust and CosmWasm best practices.
Complexity	**SUFFICIENT**	N/A

Findings Summary

Summary Title	Risk Impact	Status
Pending funds cannot be refunded to the buyer	SEVERE	RESOLVED
Pending sales created from FinalizeFor cause the NFT to be transferred incorrectly	SEVERE	RESOLVED
Reserved NFT expiration requires a full reservation period wait, causing a denial of service	SEVERE	RESOLVED
Missing denomination validation during the minimum price check	MODERATE	RESOLVED
Incorrect fee calculation in pending sale finalization	MODERATE	RESOLVED
Marketplace fees are incorrectly sent to the manager	MODERATE	RESOLVED
Exact price plugins overwrite the listing price	MODERATE	RESOLVED
Zero-price listings and rounding errors allow royalty fee bypass	LOW	PARTIALLY RESOLVED
Weak replay attack defense for pending sales	LOW	RESOLVED
Missing validation in contract initialization	LOW	RESOLVED
Two-step ownership transfer is not implemented	INFO	ACKNOWLEDGED
Redundant listing save operation	INFO	RESOLVED
Redundant config load	INFO	RESOLVED
Missing sale approvals in event emissions	INFO	RESOLVED
Inconsistent helper function usage	INFO	RESOLVED

Findings Technical Details

1. Pending funds cannot be refunded to the buyer

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

The marketplace contract implements an optional feature that requires all trades to be manually approved by the manager (i.e., `config.sale_approvals` is set to true).

In this scenario, buyer funds will be temporarily escrowed under the contract while the manager reviews and approves or rejects the trade. If the trade is rejected, the escrowed refunds will be refunded to the buyer via the `execute_reject_sale` function.

However, there are a few scenarios where the refunds will fail. Specifically, the refunds fail because the `execute_reject_sale` function dispatches an `AssetExecuteMsg::Delist` message to the asset contract and expects it to succeed:

- In `contracts/asset/src/execute/reservation.rs:96~109`, if the NFT owner manually calls `AssetExtensionExecuteMsg::UnReserve` with the `delist` boolean parameter set to true, the listing will be removed from the `asset_config.listings` state.

  This will cause the `execute_reject_sale` function to fail in `contracts/asset/src/execute/listing.rs:97` due to a `ListingNotFound` error, trapping user funds.

- In `contracts/marketplace/src/execute.rs:341`, every pending sale is only reserved for 24 hours. If the manager does not approve or reject the sale after the expiration, another buyer could create a new pending sale,

which would overwrite the reserve in the asset contract (see `contracts/asset/src/execute/reservation.rs:43~58`).

In this case, if the manager approves or rejects the latter pending sale, the former pending sale cannot be refunded to the original buyer because the listing has already been removed from the asset contract.

- In `contracts/marketplace/src/execute.rs:209`, the `execute_cancel_listing` function prevents the seller from canceling the listing when `config.sale_approvals` is true and the listing is reserved for a pending sale (i.e., `ListingStatus::Reserved`). If the manager updates the configuration to disable sale approvals and the seller cancels the listing, the pending sales refund will fail.

As a result, the buyers' funds will be stuck in the contract, causing a loss of funds.

Recommendation

Consider modifying the `execute_reject_sale` function so that `delist_msg` is dispatched only if the asset listing still exists (`asset_listing.is_ok()`), similar to the implementation in `contracts/marketplace/src/execute.rs:218~237`.

2. Pending sales created from `FinalizeFor` cause the NFT to be transferred incorrectly

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

In `contracts/marketplace/src/execute.rs:398`, the `execute_approve_sale` function uses `pending_sale.buyer` as the recipient when dispatching the `AssetExecuteMsg::Buy` message to the asset contract.

The issue is that when a user calls the `FinalizeFor` entry point with a specified `recipient` parameter in `contracts/marketplace/src/execute.rs:62`, the pending sale is created with `info.sender` as the buyer. This breaks the "buy for others" functionality because the NFT will always be sent to the buyer who paid, rather than to the `recipient` specified in the `FinalizeFor` call.

As a result, the NFT will be transferred to the sender instead of the intended recipient. If the sender is an immutable smart contract, the NFT may be permanently stuck.

Recommendation

Consider modifying the `execute_create_pending_sale` function to add a `recipient` field to the `PendingSale` struct to record the intended recipient separately from the buyer. In the `execute_approve_sale` function, use the recipient's address when constructing the `AssetExecuteMsg::Buy` message.

3. Reserved NFT expiration requires a full reservation period wait, causing a denial of service

RISK IMPACT: SEVERE	STATUS: RESOLVED

Description

When creating the listing in `contracts/marketplace/src/execute.rs:153~156`, the `execute_create_listing` function sets the reservation to the marketplace contract for one year.

This is meant only for the `reserved_for` user to call `ExecuteMsg::BuyItem` to accept the trade, not anyone else. In this case, if `config.sale_approvals` is set to true, the `execute_create_pending_sale` function will be called to dispatch an `AssetExecuteMsg::Reserve` message to the asset contract (see `contracts/marketplace/src/execute.rs:356`).

However, the transaction will fail at `contracts/asset/src/execute/reservation.rs:46` due to a `ContractError::ReservedAsset` error. This is because the asset contract still sees the marketplace contract reserving the listing for 1 year.

As a result, the reserved buyer will be unable to purchase the NFT until the one-year reservation expires.

Recommendation

Consider sending an `AssetExtensionExecuteMsg::UnReserve` message first in the `execute_create_pending_sale` function before dispatching the `AssetExecuteMsg::Reserve` message.

4. Missing denomination validation during the minimum price check

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

In `contracts/asset/src/default_plugins.rs:62`, the `min_price_plugin` function validates that the ask price amount (`ask_price.amount`) is not less than the minimum price amount (`min_price.amount`).

However, the function fails to verify that the two currencies have the same denomination. This allows sellers to bypass the minimum price requirements by providing a different ask price denom at a lower value.

Recommendation

Consider validating that the `ask_price` and `min_price` denoms are equal.

5. Incorrect fee calculation in pending sale finalization

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

In `contracts/marketplace/src/execute.rs:402`, the `execute_approve_sale` function calculates the asset price using the current fee configuration (`calculate_asset_price(pending_sale.price.clone(), config.fee_bps)`) rather than the fee stored with the listing (`Listing.price` and `Listing.asset_price`).

If the manager increases the marketplace fee after the sale is created, the `AssetExecuteMsg::Buy` message sent to the NFT contract at line 435 will use an `asset_price` amount that is less than intended, causing the transaction to revert. This is because the asset contract expects to receive the `listing.asset_price` amount defined when creating the listing in line 150.

Recommendation

Consider retrieving the `asset_price` directly from `Listing.asset_price` and computing the marketplace fee as `Listing.price - Listing.asset_price`.

6. Marketplace fees are incorrectly sent to the manager

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

In the marketplace contract, the marketplace fees are incorrectly sent to the manager address instead of the designated `fee_recipient` address.

This issue occurs in the following instances:

- `execute_buy_item` function in `contracts/marketplace/src/execute.rs:310`
- `execute_approve_sale` function in `contracts/marketplace/src/execute.rs:445`
- `execute_accept_offer` function in `contracts/marketplace/src/offers.rs:150`
- `execute_accept_collection_offer` function in `contracts/marketplace/src/offers.rs:310`

This means all marketplace fees bypass the intended fee recipient and go directly to the manager, breaking the fee distribution design that separates these roles.

Recommendation

Consider updating all marketplace fee transfer messages to use `config.fee_recipient`.

7. Exact price plugins overwrite the listing price

RISK IMPACT: MODERATE	STATUS: RESOLVED

Description

In `contracts/asset/src/plugin.rs:420`, the `ExactPrice` plugin is executed during buy operations. When this plugin runs via `run_asset_plugin`, it overwrites `ctx.data.ask_price` with the plugin's hardcoded amount in `contracts/asset/src/plugin.rs:138`.

This is problematic because it replaces the seller's intended `listing.price` set in `contracts/asset/src/plugin.rs:414`. If the listing price differs from the plugin's hardcoded amount, the validation in `contracts/asset/src/default_plugins.rs:16~27` will fail, preventing users from purchasing NFTs from the asset and marketplace contracts.

Additionally, there is a comment stating "*Exact price plugin disabled for buys for now*" at `contracts/asset/src/plugin.rs:418`. However, this is subject to the NFT contract's creator configuration and could be incorrect.

As a result, NFTs cannot be traded correctly within the asset and marketplace contract, resulting in a denial of service.

Recommendation

Consider removing the `ExactPrice` plugin from buy operations entirely.

8. Zero-price listings and rounding errors allow royalty fee bypass

RISK IMPACT: LOW	STATUS: PARTIALLY RESOLVED

Description

In `contracts/asset/src/default_plugins.rs`, the `royalty_plugin` function contains two issues that allow sellers to avoid paying royalty fees to creators:

1. In line 137, the function returns early without incurring royalty fees if the ask price amount is zero.
2. In line 156, the royalty amount is computed by the `multiply_ratio` function, which rounds down by default. For small prices or low royalty percentages, this could result in zero royalties.

As a result, royalty fees may not be charged correctly.

Recommendation

Consider implementing the following suggestions:

1. Return an error if the ask price is zero
2. Use `ceil()` when computing the royalty fee amount.

9. Weak replay attack defense for pending sales

RISK IMPACT: LOW	STATUS: RESOLVED

Description

In `contracts/marketplace/src/execute.rs:344`, the replay-attack defense for pending sales relies entirely on the asset contract reverting when the `AssetExecuteMsg::Reserve` message is executed.

For example, the function creates new pending sales without checking if unexpired pending sales already exist for the same token. This approach is dangerous as the pending sale funds are escrowed in the marketplace contract.

Recommendation

Consider implementing explicit replay defense by checking if unexpired pending sales already exist before creating a new one.

10. Missing validation in contract initialization

RISK IMPACT: LOW	STATUS: RESOLVED

Description

In `contracts/marketplace/src/state.rs:39`, the `listing_denom` field has a validation check `ensure!(!self.listing_denom.is_empty())` in the `Config` struct implementation.

However, this validation is not performed during contract initialization.

As a result, the contract could be deployed with an empty `listing_denom`, causing all subsequent listing operations to fail.

Recommendation

Consider adding validation during contract initialization to ensure that `listing_denom` is not empty.

11. Two-step ownership transfer is not implemented

RISK IMPACT: INFORMATIONAL	STATUS: ACKNOWLEDGED

Description

The codebase does not implement two-step ownership transfer in `contracts/marketplace/src/execute.rs:122`.

Using a two-step ownership transfer mechanism provides a window of opportunity for the current owner to cancel the transfer if they did not intend to initiate it or if there were unintended actions.

As a result, the ownership will be lost and cannot be recovered if transferred to an incorrect address that no one owns.

Recommendation

Consider implementing a two-step ownership transfer that proposes a new owner in the first step and requires the proposed owner to accept it as the second step.

12. Redundant listing save operation

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `contracts/asset/src/execute/sale.rs:77`, the listing is saved with `asset_config.listings.save(deps.storage, &id, &listing)`, but it is then removed at line 90, making this save operation unnecessary and wasting gas.

Recommendation

Consider removing the redundant save operation.

13. Redundant config load

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `contracts/marketplace/src/execute.rs:147`, the `config` is loaded again with `CONFIG.load(deps.storage)`, even though it was already loaded at line 137. This results in unnecessary storage access and gas costs.

Recommendation

Consider reusing the previously loaded `config` variable.

14. Missing sale approvals in event emissions

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `contracts/marketplace/src/events.rs:26`, the `update_config_event` does not emit the `sale_approvals` field, making it difficult to track this critical configuration change through event logs.

Recommendation

Consider adding `sale_approvals` as an attribute to the emitted event.

15. Inconsistent helper function usage

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

In `contracts/marketplace/src/execute.rs:231`, the `execute_cancel_listing` function manually constructs an `AssetExtensionExecuteMsg::Delist` message instead of using the available `asset_delist_msg` helper function that exists in `contracts/marketplace/src/helpers.rs:196`.

Recommendation

Consider using the `asset_delist_msg` helper function for consistency and maintainability across the codebase.

Document Control

Version	Date	Notes
-	20th November 2025	Security audit commencement date.
0.1	5th December 2025	Initial report with identified findings delivered.
0.5	14th - 15th December 2025	Fixes remediations implemented and reviewed.
1.0	15th December 2025	Audit completed, final report delivered.

Appendices

A. Appendix – Risk assessment methodology

SCV-Security employs a risk assessment methodology to evaluate vulnerabilities and identified issues. This approach involves the analysis of both the LIKELIHOOD of a security incident occurring and the potential IMPACT if such an incident were to happen. For each vulnerability, SCV-Security calculates a risk level on a scale of 5 to 1, where 5 denotes the highest likelihood or impact. Consequently, an overall risk level is derived from combining these two factors, resulting in a value from 10 to 1, with 10 signifying the most elevated level of security risk

Risk Level	Range
CRITICAL	10
SEVERE	From 9 to 8
MODERATE	From 7 to 6
LOW	From 5 to 4
INFORMATIONAL	From 3 to 1

LIKELIHOOD and **IMPACT** would be individually assessed based on the below:

Rate	LIKELIHOOD	IMPACT
5	**Extremely Likely**	Could result in severe and irreparable consequences.
4	**Likely**	May lead to substantial impact or loss.
3	**Possible**	Could cause partial impact or loss on a wide scale.
2	**Unlikely**	Might cause temporary disruptions or losses.
1	**Rare**	Could have minimal or negligible impact.

B. Appendix – Report Disclaimer

This report should not be regarded as an "endorsement" or "disapproval" of any specific project or team. These reports do not indicate the economics or value of any "product" or "asset" created by a team or project that engages SCV-Security for a security review. The audit report does not make any statements or warranties about the code's utility, safety, suitability of the business model, regulatory compliance of the business model, or any other claims regarding the fitness of the implementation for its purpose or its bug-free status. The audit documentation is intended for discussion purposes only. The content of this audit report is provided "as is," without representations and warranties of any kind, and SCV-Security disclaims any liability for damages arising from or in connection with this audit report. Copyright of this report remains with SCV-Security.

THANK YOU FOR CHOOSING



SCV SECURITY

scv.services

contact@scv.services